Einstein Summation as an Architecture Language

MLPs, CNNs, GNNs, LSTMs, and Transformers in `torch.einsum`

DeepLearningSystems-2026

From index equations to efficient kernels

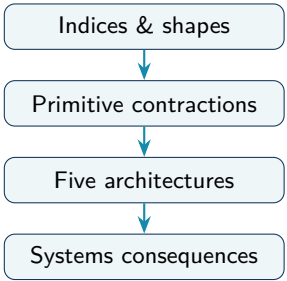
2026

One notation for expressing computation, checking shapes, and reasoning about systems costs.

Learning objectives and route

By the end, you can:

- read free and contracted indices;
- translate equations into `torch.einsum`;
- derive five common architectures;
- predict FLOPs, intermediates, and fusion opportunities;
- test an expression with explicit shapes.



Einstein notation: indices carry the specification

$$C_{ij} = \sum_{k} A_{ik} B_{kj}$$

Two roles

- **Free indices** i, j survive in the output.
- **Contracted index** k appears in inputs but not output, so it is summed.

Shape contract

$A : [I, K]$, $B : [K, J]$, $C : [I, J]$.
Repeated labels must have compatible extent.
Output label order fixes output axis order.

```
torch.einsum('ik,kj->ij', A, B)
```

From nested loops to one contraction

Literal algorithm

```
for i in range(I):
  for j in range(J):
    C[i,j] = 0
    for k in range(K):
      C[i,j] += A[i,k]*B[k,j]
```

Declarative algorithm

```
ik,kj->ij
```

The labels state:

- iteration space: i, j, k;
- reduction: k;
- output layout: i, j.

Important

Einsum specifies the result, not a unique loop order or kernel. The implementation still chooses tiling, fusion, and contraction order.

The einsum grammar: keep, sum, broadcast, diagonalize

Operation	Equation / subscript	Output shape
Dot product	$x_i y_i$ / `'i,i->'`	scalar
Outer product	$x_i y_j$ / `'i,j->ij'`	$[I, J]$
Batch matmul	$A_{bik} B_{bkj}$ / `'bik,bkj->bij'`	$[B, I, J]$
Broadcast weights	$X_{bti} W_{io}$ / `'bti,io->bto'`	$[B, T, O]$
Trace	A_{ii} / `'ii->'`	scalar
Diagonal	A_{ii} / `'ii->i'`	$[I]$

Rule of thumb

A label repeated within one operand selects a diagonal. A label absent after `->` is reduced. Ellipsis `...` denotes unnamed broadcast dimensions.

Shape discipline: a four-step correctness test

1. **Name every axis**: e.g., batch b, token t, feature d.
2. **Write operand shapes**: $X : [B, T, D]$, $W : [D, O]$.
3. **Check each shared label**: every d must mean the same extent D.
4. **Read output labels**: `btd,do->bto` gives $[B, T, O]$.

Executable shape assertion

```
B,T,D,O = 2,7,16,32
x = torch.randn(B,T,D); w = torch.randn(D,O)
y = torch.einsum('btd,do->bto', x, w)
assert y.shape == (B,T,O)
```

Frequent bug

Reusing a letter for different semantic axes may silently compute the wrong result when the extents happen to match.

MLP: an affine layer is a batched contraction

For $X : [B, I]$, $W : [I, O]$, $a : [O]$:

$$Z_{bo} = \sum_i X_{bi} W_{io} + a_o, \qquad H_{bo} = \phi(Z_{bo}).$$

```
Z = torch.einsum('bi,io->bo', X, W) + a
```

What einsum covers

The dense linear map and any leading batch-like axes.

What it does not cover

Bias addition and nonlinear activation are elementwise operations, often fused by a compiler.

Cost

Approximately $2BIO$ floating-point operations; weights contribute IO parameters.

MLP over tokens and stacked experts

Token-wise projection

$X : [B, T, D]$, $W : [D, F]$:

$$H_{btf} = \sum_d X_{btd} W_{df}, \qquad \texttt{btd,df->btf}.$$

Weights are reused independently at every b, t.

Expert-specific projection

$W : [E, D, F]$ gives $H_{btef} = \sum_d X_{btd} W_{edf}$, or `btd,edf->btef`. This materializes all expert outputs.

Systems question

Materialize $[B, T, E, F]$, or gather routed tokens and issue grouped matrix multiplies?

CNN: expose patches, then contract

Let patches $P : [B, H_o, W_o, C_i, K_h, K_w]$ and kernels $K : [C_o, C_i, K_h, K_w]$.

$$Y_{bhwo} = \sum_{cuv} P_{bhwcuv} K_{ocuv}$$

```
Y = torch.einsum('bhwcuv,ocuv->bhwo', P, K)
```

Index meaning

b: batch; h, w: output location; c: input channel; u, v: kernel offset; o: output channel.

Equivalent view

Flatten (c, u, v) into one reduction axis. Convolution becomes a matrix multiplication after *im2col*.

CNN systems view: arithmetic versus movement

Explicit patches

- Simple GEMM lowering.
- Patch tensor size: $BH_oW_oC_iK_hK_w$.
- Repeats overlapping input pixels.

Implicit / direct convolution

- Generate patch addresses on demand.
- Tile input and kernels in fast memory.
- Avoid full im2col materialization.

Compute count

$\approx 2BH_oW_oC_oC_iK_hK_w$ FLOPs. Performance depends on reuse and layout, not only this count.

Mini-check

If $K_h = K_w = 1$, which indices disappear? The operation reduces to a per-pixel channel projection: `bhwc,oc->bhwo`.

GNN: neighborhood aggregation as a contraction

For dense adjacency $A : [N, N]$ and node features $X : [N, D]$:

$$M_{id} = \sum_j \widehat{A}_{ij} X_{jd}, \qquad \texttt{ij,jd->id}.$$

Then apply a learned projection $W : [D, F]$:

$$H_{if} = \sum_d M_{id} W_{df}, \qquad \texttt{id,df->if}.$$

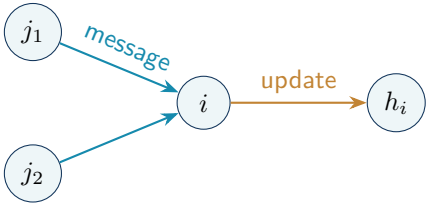
Combined equation

$$H_{if} = \sum_{j,d} \widehat{A}_{ij} X_{jd} W_{df}, \qquad \texttt{ij,jd,df->if}.$$

Reality check

Real graphs are sparse: use scatter/segment reductions or sparse matrix multiply, not dense $N \times N$ adjacency.

GNN layer: semantics and execution



1. Gather source features X_j.
2. Form edge messages.
3. Reduce messages by destination i.
4. Apply projection / nonlinearity.

Contraction-order choice

$(\widehat{A}X)W$ costs aggregation in width D; $\widehat{A}(XW)$ costs aggregation in width F. Choose based on D, F, sparsity, and reuse. Associativity preserves the linear result.

LSTM: fuse four gate projections

At time t, concatenate input and previous hidden state conceptually:

$$G_{brg} = \sum_d S_{bd} W_{drg} + a_{rg}, \quad S = [x_t; h_{t-1}], \quad r \in \{i, f, o, g\}.$$

Shapes: $S : [B, D]$, $W : [D, 4, H]$, $G : [B, 4, H]$.

```
G = torch.einsum('bd,drh->brh', S, W) + a
```

Why fuse?

One larger matrix multiplication reuses S, reduces launch overhead, and usually outperforms four separate projections.

Notation warning

Here r is the gate axis and h is the hidden-feature axis; do not reuse h for the hidden-state tensor name inside the index equation.

LSTM recurrence: contraction plus elementwise state update

Split G along its gate axis and apply nonlinearities:

$$i_t = \sigma(G_i), \quad f_t = \sigma(G_f), \quad o_t = \sigma(G_o), \quad \widetilde{c}_t = \tanh(G_g).$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t).$$

Parallel within a step

All four affine gate computations are one contraction.

Sequential across time

(h_t, c_t) depends on (h_{t-1}, c_{t-1}); sequence steps form a critical path.

Systems implication

Kernel fusion helps inside each step, but recurrence limits time-axis parallelism; scan methods trade extra work for parallel depth.

Transformer I: project tokens into heads

Input $X : [B, T, D]$; projection weights $W^Q, W^K, W^V : [D, H, P]$.

$$Q_{bthp} = \sum_d X_{btd} W^Q_{dhp}$$

```
Q = torch.einsum('btd,dhp->bthp', X, Wq)
```

Analogously obtain $K, V : [B, T, H, P]$. Often the three weights are packed:

$$R_{btzhp} = \sum_d X_{btd} W_{zdhp}, \qquad z \in \{Q, K, V\}.$$

```
R = torch.einsum('btd,zdhp->btzhp', X, Wqkv)
```

Implementation note

Production kernels commonly flatten (H, P) and use GEMM; the explicit head labels remain valuable for reasoning and tests.

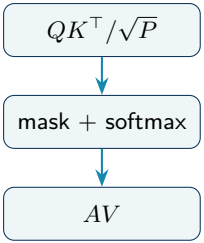
Transformer II: attention scores and weighted values

$$S_{bhts} = \frac{1}{\sqrt{P}} \sum_p Q_{bthp} K_{bshp}$$

```
S = torch.einsum('bthp,bshp->bhts', Q, K) /
sqrt(P)
```

$$A = \mathrm{softmax}_s(S + M), \qquad O_{bthp} = \sum_s A_{bhts} V_{bshp}.$$

```
O = torch.einsum('bhts,bshp->bthp', A, V)
```



Transformer III: merge heads and project

Attention output $O : [B, T, H, P]$ and output weight $W^O : [H, P, D]$:

$$Y_{btd} = \sum_{h,p} O_{bthp} W^O_{hpd}.$$

```
Y = torch.einsum('bthp,hpd->btd', O, Wo)
```

Full attention tensor shapes

X	Q, K, V	S, A	O	Y
$[B, T, D]$	$[B, T, H, P]$	$[B, H, T, T]$	$[B, T, H, P]$	$[B, T, D]$

Memory bottleneck

Materializing A costs $O(BHT^2)$ elements. FlashAttention tiles the score computation and online softmax so the full matrix need not reside in high-bandwidth memory.

Systems lens: equivalent equations, unequal executions

Decision	Question	Typical response
Contraction order	Which intermediate is smallest?	Reassociate linear contractions; use an optimizer for ≥ 3 operands.
Layout	Are reduction axes contiguous?	Permute/pack once, then feed GEMM-friendly tiles.
Materialization	Is an intermediate reused enough?	Fuse producer–consumer or recompute cheap values.
Sparsity	Are most coefficients zero?	Store edges/blocks; use segmented or sparse kernels.
Precision	What range/accuracy is required?	Low-precision inputs, wider accumulation, scaling.
Batching	Are shapes regular?	Pad/bucket, grouped GEMM, or shape-specialized kernels.

Roofline reminder

Speed is bounded by $\min(\text{peak FLOP/s},\ \text{bandwidth} \times \text{arithmetic intensity})$. Einsum reveals reuse opportunities but does not guarantee they are realized.

Mini-exercises: predict before you run

1. $X : [8, 128, 64]$, $W : [64, 256]$. Write the token-wise projection and output shape.
2. For `bik,bkj->bij`, which index is reduced? What are the compatibility constraints?
3. Rewrite a 1×1 convolution for NHWC input as an einsum.
4. In dense GNN propagation, compare $(AX)W$ with $A(XW)$ when $D = 1024$ and $F = 64$.
5. For attention with $B = 4, H = 8, T = 2048$, how many elements are in the score tensor?

Answers

1. `btd,df->btf`, shape $[8, 128, 256]$.
2. k; batch b and contracted extent k must agree.
3. `bhwc,oc->bhwo`.
4. Project first to aggregate 64-wide features, subject to sparsity/kernel details.
5. $4 \cdot 8 \cdot 2048^2 = 134{,}217{,}728$ elements (about 256 MiB in FP16).

Takeaways and references

Five durable habits

1. Name axes by meaning.
2. Distinguish free from reduced indices.
3. Assert shapes with tiny tensors.
4. Separate mathematical equivalence from execution cost.
5. Inspect intermediates, layout, and memory traffic.

Further reading

- PyTorch, *torch.einsum documentation*.
- Kolda & Bader, *Tensor Decompositions and Applications*, 2009.
- Vaswani et al., *Attention Is All You Need*, 2017.
- Dao et al., *FlashAttention*, 2022.
- Hamilton, *Graph Representation Learning*, 2020.

Indices are a compact architecture specification.